

defineback

November 15, 2001
15:48

Contents

1	A Program for Defining 2-D Plasmas	1
2	Example Input File	2
3	Plasma Data	3
4	Source Group Data	5
5	Other Considerations	6
6	INDEX	26

1 A Program for Defining 2-D Plasmas

This program is designed to be used in conjunction with **definegeometry2d**. Like **definegeometry2d**, the bulk of the data details are provided in one or more text files that can be generated by some other code. The data formats used here are sufficiently simple that they can also be produced manually. The specification of the “background” consists of two pieces:

1. The densities, velocities, and temperatures of the background (plasma) species. For simplicity, we will refer to these as the “plasma” species, even though in some problems, these species may be electrically neutral (e.g., for the purposes of studying neutral-neutral interactions).
2. A description of the neutral sources (e.g., recycling of a plasma flow onto a material surface).

This initial version of **defineback** is still “rough around the edges”. While many opportunities for improvement are apparent, the most promising prospect is to incorporate this functionality into a graphical tool that also carries out the geometry definition. What comes next will be determined by project needs and manpower availability; stay tuned.

As in **definegeometry2d**, the input file for **defineback** is not listed in **degas2.in**. Again, the reason is that this code is, in principle, just one of many different ways to provide background data to **DEGAS 2**.

The command line for the execution of **defineback** specifies the name of the main **defineback** input file. This file will contain pointers to other files that provide the plasma and source information.

defineback an_input_file

Like other text input files used in **DEGAS 2**, blank lines, spaces, and lines beginning with a comment character **#** are ignored (comments can also appear at the end of a line).

2 Example Input File

Here is an example input file:

```
plasma_file mrx_plasma3
```

```
new_source_group
  source_type puff
  source_geom surface
  source_species D2
  source_root_sp D2
  source_nflights 1000
  specify_flux
  source_file mrx_source_2.in
end_source_group
```

```
new_source_group
  source_type puff
  source_geom surface
  source_species D2
  source_root_sp D2
  source_nflights 1000
  specify_flux
  source_stratum     1
  source_segment     *
  source_strength    1.e20
end_source_group
```

3 Plasma Data

This program contains by default a rather general mechanism for specifying the plasma data via a text file.

As in the above example, there is essentially only one keyword associated with specifying the plasma data, *plasma_file*. In the default implementation, this keyword has a single argument consisting of the name of the file containing the plasma data to be read in.

The default implementation requires the plasma data to be in a tabular format (with space or tab delimiters, just like any other **DEGAS 2** text input file). The key assumption for this routine is that all data pertaining to a particular zone are contained on a single line of the file. If the zone number is provided as the first number of a line, it will be used in assigning the data values on that line; in principle, any ordering of the zones could be used. Note that this zone number is the “absolute” zone, not the plasma zone number. If the zone number is not specified, the line number is used as the zone number. Problems having both the geometry and plasma data specified by some other code make this approach particularly simple.

A header line at the top of the file informs the code of the ordering of the data on each line. For example,

zone	T(1)	N(1)	T(2)	N(2)
1	1.0000000475e-03	1.0000000000e+01	1.0000000475e-03	1.0000000000e+01
2	3.0000000000e+00	3.0000000000e+19	3.0000000000e+00	3.0000000000e+19
3	1.0000000475e-03	1.0000000000e+01	1.0000000475e-03	1.0000000000e+01

Here, the first column contains the zone number. The second and third columns give the temperature and density of the first background species (referring to their ordering in the input file provided for **problemsetup**). The last two columns have the temperature and density for the second background species. Note that the assumed units are eV for temperatures and m^{-3} for densities. Velocities (in meters per second) would be indicated by a string like **V2(1)** to indicate the velocity in the x_2 (e.g., y) direction for the first background species. The columns may be in any order. The strings can be either lower or upper case (e.g., **ZONE** or **n(1)** would also be recognized). If particular data are not provided (such as the velocities in the above example), default values of 0 will be used.

There is also a keyword *plasma_coords* allowing the user to specify the coordinate system of the plasma velocities as either *cartesian* or *cylindrical*. In two-dimensional problems, this choice is governed by the symmetry specified in **definegeometry2d**. As such, the user can ignore this keyword. It is provided only for completeness.

The *get_n_t* subroutine that reads the plasma file has a simple, but flexible interface consisting of the string that follows the *plasma_file* keyword in the **defineback** input file. As such, the user can easily replace this subroutine with a more sophisticated piece of code that can adapt available data to the geometry of a particular case. For example, some or even all of the plasma zones may be generated by **definegeometry2d**. If the user has some notion of the spatial variation of the plasma data, a specific version of the *get_n_t* subroutine can be written to map that description on to the **DEGAS 2** geometry using the internal geometry arrays that are available via the common blocks.

A more general implementation might pack several arguments into the string following the *plasma_file* keyword. For example, the string might contain multiple file names, as well as integer and floating point parameters. The string would be parsed in whatever manner is desired (although use of **DEGAS 2**'s string manipulation routines is recommended).

An example of such a subroutine, used in work on the National Compact Stellarator Experiment (NCSX), is included here as an example. The version of the *get_n_t* subroutine to be used is controlled by the **GEOMETRY** macro at the top of the file. The default behavior is obtained by setting it to **DEFAULT** before compiling. If set to **NCSX**, the specialized routine designated with the **FWEB** tag "Plasma Data for NCSX" is invoked. If your problem needs such additional processing, you can either replace this subroutine or add it to the end of the file. In the latter case, you will need to define an additional **FWEB** macro variable and tag to force it to be used when **ftangle** is run.

The **NCSX** example shows how the stratum label available in **definegeometry2d** can be used to locate the zones generated by the triangulation of a particular polygon specified in a **definegeometry2d** input file. Namely, the polygons in the **NCSX** problem were drawn so that plasma parameters would be constant over them (they corresponded to magnetic flux surfaces); each of the polygons was assigned a unique stratum number. That information is available in the polygon netCDF file generated by **definegeometry2d** (the *g2_polygon_stratum* array); the zone numbers associated with each polygon are also there (in *g2_polygon_zone*). The two arrays are combined in this *get_n_t* subroutine to form a mapping array, *zone_stratum*, that provides the stratum number associated with each zone. This then allows the subroutine to determine the plasma parameters to be assigned to those zones.

4 Source Group Data

The rest of the input file describes the sources to be used in the run. As the above example input file shows, each source group is specified in its own section. The specification is begun with a *new_source_group* command and ended with *end_source_group*. The order of the keywords in between *should not* matter.

Each of the keywords occupies a line of the form:

keyword arguments

with some keywords having multiple arguments, others having none at all. Lower case single letters (perhaps with subscripts) are used to represent integer arguments. Upper case single letters correspond to real (i.e., floating point) arguments. All other arguments are strings.

source_type **typ** gives the type of the source. The only types currently treated in DEGAS 2 are **typ = plate**, **puff**, and **recomb**. The last is not needed in a **defineback** input file since a recombination source is established automatically by the inclusion of the appropriate reaction in the **problemsetup** input file.

source_geom **geom** specifies the geometry of the source. The only valid value in **defineback** at this time is **geom = surface** (e.g., a line in a poloidal plane of an axisymmetric torus). However, there is no default set, so this must be included with each source group. Other options will be implemented as needed.

source_species **sym** determines the species to be generated by the source. This species must be in the list of test species for the problem. The **sym** is the species symbol that appears in the *species_infile*.

source_root_sp **sym** determines the species underlying the source. For a *plate* source, this would be the ion corresponding to the neutral *source_species*. For a *puff* source, this would be the same as *source_species*. The code currently requires that *source_root_sp* be specified. Eventually, code will be added to infer a reasonable default value from the other information provided.

source_nflights n_{flights} overrides a default value of 100 flights per source group. This keyword allows the user to avoid having to edit the background netCDF file to set the number of flights.

specify_flux informs the code that the *source_strength* will represent a flux of particles (particles per square meter per second). This is the default.

specify_current informs the code that the *source_strength* will represent a current of particles (particles per second).

source_stratum $i_1 i_2 i_3 \dots$ is used together with the *source_segment* and *source_strength* keywords to explicitly specify the location and strength of a number (preferably a small number) of source surfaces. These three keywords must all have the same number of arguments. The keywords and their arguments have no preferred order. The value of *source_stratum* corresponds to the *strata* associated with a sector, e.g., as assigned with the *stratum* keyword in **definegeometry2d**.

source_segment $i_1 i_2 i_3 \dots$ identifies which segments of the stratum specified by *source_stratum* are to be used. If the *stratum* values have been chosen judiciously in running *definegeometry2d*, the user should be able to infer the desired segment values by counting off the segments of the corresponding polygon in the *definegeometry2d* input file (with the first side of the polygon being segment 0, the second 1, etc.). If all of the segments of a stratum are to be used, with the same source strength, a single wild-card argument of ***** can be given with *source_segment*; *source_stratum* and *source_strength* should have only one argument in this case.

source_strength S_1 S_2 S_3 ... gives the source strength (with units determined by the appearance of either the *specify_flux* or *specify_current* keywords) to be used in conjunction with *source_stratum* and *source_segment*.

source_file **filename** is an alternative method of specifying the location and strength of the source (i.e., in lieu of using *source_stratum*, *source_segment*, and *source_strength*). The file **filename** is a text file of the usual type in which blank lines and comment lines beginning with # are ignored. Each line specifies a source segment:

i_{stratum} i_{segment} S_{strength}

That is, the stratum and segment numbers followed by its strength. This is the only permissible order for the three parameters. No wild cards are allowed. The units of the source strength are determined by which of *specify_flux* and *specify_current* is in effect.

5 Other Considerations

If a plate source is specified, a default sheath physics model is implemented. The *target_e_ion_mult* array is set to 1, implying that the source ion energy will start at 1 times the thermal energy corresponding to the local ion temperature. A fixed sheath potential of $3T_e$ is assumed (the code will expect the electron species to be the first background species; again, this is not intended to be a refined implementation). The sheath potential is placed into the *target_e_ion_delta*, representing a constant (for each source particle starting at that segment) increment to the source ion's energy.

\$Id: \$

"defineback.f" 1 ≡

@m FILE 'defineback.web'

The main program.

```
"defineback.f" 5.1 ≡
program defineback
  implicit_none_f77
  implicit_none_f90

  integer nargs
  character*FILELEN inputfile

  sy_decls

  nargs = arg_count()
  if (nargs ≠ 1) then
    assert('_Command_line_must_specify_an_input_file' = '_')
  end if
  call command_arg(1, inputfile)

  call readfilenames
  call read_geometry

  call nc_read_species
  call nc_read_reactions
  call nc_read_materials
  call nc_read_pmi
  call nc_read_problem

  call setup_back_arrays
  call setup_background(inputfile)
  call erase_geometry

  @if 0
    call mem_check
  @endif

  stop
end

⟨ Functions and subroutines 5.2 ⟩
```


Setup background.

```
"defineback.f" 5.2 ≡
  @m DEFAULT 0    // Reads plasma data from a file
  @m NCSX 1    // Plasma data computed using data from 3 files
  @m GEOMETRY DEFAULT
  @m open_file(aunit, aname)
    open(unit = aunit, file = aname, status = 'old', form = 'formatted', iostat = open_stat)
    assert(open_stat ≡ 0)
  @m next_line #:0
  @m input_done #:0
  @m next_stratum #:0
  @m strata_done #:0
  @m next_segment #:0
  @m segments_done #:0
  @m next_strength #:0
  @m strengths_done #:0
  @m next_srcline #:0
  @m srcfile_done #:0
  @m increment_iseg iseg++
  if (iseg > dim_segments) then
    dim_segments = iseg
    var_realloca(source_stratum)
    var_realloca(source_segment)
    var_realloca(all_segments)
    var_realloca(source_strength)
    source_stratum_iseg = int_uninit
    source_segment_iseg = int_uninit
    all_segments_iseg = FALSE
    source_strength_iseg = real_uninit
  end if

⟨ Functions and subroutines 5.2 ⟩ ≡
  subroutine setup_background(inputfile)

    define_dimen(segment_ind, dim_segments)
    define_varp(source_strength, FLOAT, segment_ind)
    define_varp(source_stratum, INT, segment_ind)
    define_varp(source_segment, INT, segment_ind)
    define_varp(all_segments, INT, segment_ind)

    implicit_none_f77
    gi_common    // Common
    zn_common
    bk_common
    pr_common
    so_common
    sc_common
    sp_common
    implicit_none_f90

    character*FILELEN inputfile    // Input
    real density, temperature, dummy, circum, area    // Local
```

```

integer type, geom, nflights, length, p, b, e, nunit_n, specify_flux, nunit_t, num_segments, iseg, i,
         uedge, open_stat, targ, disk2, dim_segments

character*1 back_label
character*LINELEN line, type_string, nt_string
character*FILELEN source_file

sp_decl(species)
sp_decl(root_sp)
bk_decl(back)
zn_decl(zone)
sc_decl(sector)
vc_decl(xdiff)

declare_varp(source_strength)
declare_varp(source_stratum)
declare_varp(source_segment)
declare_varp(all_segments)

⟨ Memory allocation interface 0 ⟩
st_decls

open_file(diskin, inputfile)
so_set_run_flags
so_grps = 0
so_seg_tot = 0
dim_segments = 0     /* Use this as a default since these problems are all likely to be 2-D so that the
        velocity transformations are governed by geometry_symmetry. */
if (geometry_symmetry  $\equiv$  geometry_symmetry_plane) then
    background_coords = plasma_coords_cartesian
else if (geometry_symmetry  $\equiv$  geometry_symmetry_cylindrical) then
    background_coords = plasma_coords_cylindrical
else
    assert('Unexpected_value_of_geometry_symmetry'  $\equiv$  '␣')
end if     /* Loop to read through input file. */
next_line: continue

if ( $\neg$ read_string(diskin, line, length)) then
    close (unit = diskin)
    goto input_done
else
    assert(length  $\leq$  len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))

    if (line(b : e)  $\equiv$  'plasma_file') then
        assert(next_token(line, b, e, p))
        nt_string = line(b :)
        call get_n_t(nt_string)
    else if (line(b : e)  $\equiv$  'plasma_coords') then
        assert(next_token(line, b, e, p))
        if (line(b : e)  $\equiv$  'cartesian') then
            background_coords = plasma_coords_cartesian
        else if (line(b : e)  $\equiv$  'cylindrical') then
            background_coords = plasma_coords_cylindrical

```

```

else
    assert('␣Illegal␣value␣of␣plasma␣coords' ≡ '␣')
end if

else if (line(b : e) ≡ 'new_source_group') then
    so_grps++
    type = int_undef
    geom = int_undef
    species = int_undef
    root_sp = int_undef
    nflights = 100
    specify_flux = TRUE
    num_segments = int_uninit    /* We can initialize these arrays out to their current dimensions.
        If they get expanded during the process of adding segments, the additional initializations
        will be done segment-by-segment (see increment_iseg). */
    if (dim_segments > 0) then
        do iseg = 1, dim_segments
            source_stratum_iseg = int_uninit
            source_segment_iseg = int_uninit
            all_segments_iseg = FALSE
            source_strength_iseg = real_uninit
        end do
    end if

else if (line(b : e) ≡ 'source_type') then
    assert(next_token(line, b, e, p))
    type_string = line(b : e)
    assert(type ≡ int_undef)
    do i = 1, so_type_num
        if (type_string ≡ so_name(i)) then
            type = i
        end if
    end do
    assert(type ≠ int_undef)    // Also do a final check before using

else if (line(b : e) ≡ 'source_geom') then
    assert(next_token(line, b, e, p))
    assert(geom ≡ int_undef)
    if (line(b : e) ≡ 'point') then
        geom = so_point
    else if (line(b : e) ≡ 'line') then
        geom = so_line
    else if (line(b : e) ≡ 'surface') then
        geom = so_surface
    else if (line(b : e) ≡ 'volume') then
        geom = so_volume
    end if
    assert(geom ≠ int_undef)

else if (line(b : e) ≡ 'source_species') then
    assert(next_token(line, b, e, p))
    assert(species ≡ int_undef)
    species = sp_lookup(line(b : e))
    assert(sp_check(species))

```

```

    else if (line(b : e) ≡ 'source_root_sp') then
        assert(next_token(line, b, e, p))
        assert(root_sp ≡ int_undef)
        root_sp = sp_lookup(line(b : e))
        assert(sp_check(root_sp))

    else if (line(b : e) ≡ 'source_nflights') then
        assert(next_token(line, b, e, p))
        nflights = read_integer(line(b : e))

    else if (line(b : e) ≡ 'specify_flux') then
        specify_flux = TRUE

    else if (line(b : e) ≡ 'specify_current') then
        specify_flux = FALSE

    else if (line(b : e) ≡ 'source_stratum') then
        iseg = 0
next_stratum: continue
        if (¬next_token(line, b, e, p)) then
            if (num_segments ≡ int_uninit) then
                num_segments = iseg
            else
                assert(num_segments ≡ iseg)
            end if
            goto strata_done
        else
            increment_iseg
            source_stratumiseg = read_integer(line(b : e))
        end if
        goto next_stratum
    strata_done: continue

    else if (line(b : e) ≡ 'source_segment') then
        iseg = 0
next_segment: continue
        if (¬next_token(line, b, e, p)) then
            if (num_segments ≡ int_uninit) then
                num_segments = iseg
            else
                assert(num_segments ≡ iseg)
            end if
            goto segments_done
        else
            increment_iseg
            if (line(b : e) ≡ '*') then
                all_segmentsiseg = TRUE
                source_segmentiseg = int_unused
            else
                all_segmentsiseg = FALSE
                source_segmentiseg = read_integer(line(b : e))
            end if
        end if
        goto next_segment
    segments_done: continue

```

```

        else if (line(b : e) ≡ 'source_strength') then
            iseg = 0
next_strength: continue
            if (¬next_token(line, b, e, p)) then
                if (num_segments ≡ int_uninit) then
                    num_segments = iseg
                else
                    assert(num_segments ≡ iseg)
                end if
                goto strengths_done
            else
                increment_iseg
                source_strength_iseg = read_real(line(b : e))
            end if
            goto next_strength
strengths_done: continue

        else if (line(b : e) ≡ 'source_file') then
            iseg = 0
            assert(next_token(line, b, e, p))
            source_file = line(b : e)
            disk2 = disk + 1
            open_file(disk2, source_file)
next_srcline: continue
            if (¬read_string(disk2, line, length)) then /* I suppose we could generalize this if-then
                to match the others above and add a format specification below so that this file could
                contain just a subset of stratum, segment, and strength. But, do not see the need. */
                assert(num_segments ≡ int_uninit)
                num_segments = iseg
                close(unit = disk2)
                goto srcfile_done
            else
                assert(length ≤ len(line))
                length = parse_string(line(: length))
                p = 0
                increment_iseg
                assert(next_token(line, b, e, p))
                source_stratum_iseg = read_integer(line(b : e))
                assert(next_token(line, b, e, p))
                source_segment_iseg = read_integer(line(b : e))
                assert(next_token(line, b, e, p))
                source_strength_iseg = read_real(line(b : e))
            end if
            goto next_srcline
srcfile_done: continue

        else if (line(b : e) ≡ 'end_source_group') then
            ⟨ Setup Source Group 5.3 ⟩
        else
            assert('Unknown_keyword' ≡ line(b : e))
        end if
    end if
    goto next_line

```

input_done: **continue**

```

    /* Write densities and temperatures out to a file designed to mimic format of outputbrowser text
       files so these can be used with the AVS/Express graphing package. */
    nunit_n = disk_n + 5
    open(unit = nunit_n, file = 'density_1.txt', status = 'unknown')
    write(nunit_n, '(i4.4)') zn_num
    do back = 1, bk_num
        nunit_t = nunit_n + 1
        write(back_label, '(i1)') back
        open(unit = nunit_t, file = 'temperature_' || back_label || '.txt', status = 'unknown')
        write(nunit_t, '(i4.4)') zn_num
        do zone = 1, zn_num
            if (zn_type(zone) == zn_plasma) then
                density = bk_n(back, zone)
                temperature = bk_temp(back, zone)
            else
                density = zero
                temperature = zero
            end if
            if (back == 1) then
                write(nunit_n, '(4x,i4,6x,i4,6x,1pe13.5,2x,0pf7.4)') zone, back, density, dummy
            end if
            write(nunit_t,
                '(4x,i4,6x,i4,6x,1pe13.5,2x,0pf7.4)') zone, back, temperature/electron_charge,
                dummy
        end do
        close(unit = nunit_t)
    end do
    close(unit = nunit_n)

    var_reallocb(source_base_ptr)
    var_reallocb(source_num_segments)
    var_reallocb(source_type)
    var_reallocb(source_geometry)
    var_reallocb(source_num_flights)
    var_reallocb(source_num_checkpoints)
    var_reallocb(source_species)
    var_reallocb(source_root_species)
    var_reallocb(source_total_current)
    var_reallocb(source_weight_norm)
    var_reallocb(source_scale_factor)

    var_reallocb(source_current)
    var_reallocb(source_segment_ptr)
    var_reallocb(source_segment_rel_wt)
    var_reallocb(source_segment_prob_alias)
    var_reallocb(source_segment_ptr_alias)

    var_reallocb(source_strength)
    var_reallocb(source_stratum)
    var_reallocb(source_segment)
    var_reallocb(all_segments)

    if (so_seg_tot == 0) then

```

```
    assert('No_sources_were_defined!' ≡ '')
end if

call set_background_sources

uedge = FALSE
call init_wt_alias(uedge)

call write_background
    /* Updated plasma sector properties and possibly relabeled vacuum zones. */
call write_geometry

var_free(source_strength)
var_free(source_stratum)
var_free(source_segment)
var_free(all_segments)

return
end
```

See also section 5.4.

This code is used in section 5.1.

Setup a source group.

```

⟨ Setup Source Group 5.3 ⟩ ≡
  assert(num_segments > 0)
  var_realloca(source_base_ptr)
  var_realloca(source_num_segments)
  var_realloca(source_type)
  var_realloca(source_geometry)
  var_realloca(source_num_flights)
  var_realloca(source_num_checkpoints)
  var_realloca(source_species)
  var_realloca(source_root_species)
  var_realloca(source_total_current)
  var_realloca(source_weight_norm)
  var_realloca(source_scale_factor)
  so_base(so_grps) = so_seg_tot + 1
  assert(type ≠ int_undef)
  so_type(so_grps) = type
  assert(geom ≠ int_undef)
  so_geom(so_grps) = geom
  so_nflights(so_grps) = nflights
  so_chkpt(so_grps) = 0
  assert(sp_check(species))
  so_species(so_grps) = species     // Use neutralize_species?
  assert(sp_check(root_sp))
  so_root_sp(so_grps) = root_sp
  so_tot_curr(so_grps) = zero
  so_scale(so_grps) = one
  do sector = 1, nsectors
    do iseg = 1, num_segments
      if ((strata_sector ≡ source_stratum_iseg) ∧ ((sector_strata_segment_sector ≡
        source_segment_iseg) ∨ all_segments_iseg ≡ TRUE)) then
        targ = sector_type_pointer_sector,sc_target
        zone = sector_zone_sector
        assert(sc_target_check(targ))
        so_seg_tot++
        var_realloca(source_current)
        var_realloca(source_segment_ptr)
        var_realloca(source_segment_rel_wt)
        var_realloca(source_segment_prob_alias)
        var_realloca(source_segment_ptr_alias)
        assert(source_strength_iseg > zero ∧ source_strength_iseg ≠ real_uninit)
        if (specify_flux ≡ TRUE) then
          vc_difference(sector_points_sector,sc_neg, sector_points_sector,sc_pos, xdiff)
          if (geometry_symmetry ≡ geometry_symmetry_plane) then     /* Note: in a 3-D problem, the
            physical length to be used here would typically be smaller than this. But, by definition
            of this symmetry the problem spans the full length of the universal cell in this direction.
            */
            circum = universal_cell_max2 - universal_cell_min2
          else if (geometry_symmetry ≡ geometry_symmetry_cylindrical) then
            circum = PI * (sector_points_sector,sc_neg,1 + sector_points_sector,sc_pos,1)
          else
            assert('Unexpected_geometry_symmetry_in_setting_up_source' ≡ '␣')

```



```

    end if
    area = circum * vc_abs(xdiff)
    assert(area > zero)
    source_currentso_seg_tot = source_strengthiseg * area
else /* Specify current */
    source_currentso_seg_tot = source_strengthiseg
end if
so_tot_curr(so_grps) += source_currentso_seg_tot
source_segment_ptrso_seg_tot = sector
source_segment_rel_wtso_seg_tot = one
/* Assign sheath properties using the old DEGAS model since we lack specific boundary
parameters (and fluid velocities). An alternative (e.g., in the UEDGE case) would be to use
the inc_ion_energy function. Assumes that the first background species is "e". */
if (so_type(so_grps) == so_plate) then
    assert(sp_sy(pr_background(1)) == 'e')
    assert(bk_temp(1, zone) > zero)
    target_e_ion_deltatarg = const(3.) * bk_temp(1, zone)
    target_e_ion_multtarg = one
end if
end if
end do
end do
so_nseg(so_grps) = so_seg_tot - so_base(so_grps) + 1

```

This code is used in section 5.2.

Decide on which geometry specific code to use.

⟨Functions and subroutines 5.2⟩ +≡

@#if (*GEOMETRY* == *DEFAULT*)
 ⟨Read Plasma From File 5.5⟩

@#elif (*GEOMETRY* == *NCSX*)
 ⟨Plasma Data For NCSX 5.7⟩

@#endif

Find background densities, temperatures, and flow velocities. These are read from a file using a rather flexible, user-specified format. The key assumption for this routine (conceivably others could be developed) is that all data pertaining to a particular zone are contained on a single line of the file. If the zone number is provided as the first number of a line, it will be used in assigning the data values on that line; in principle, the any ordering of zones could be used. Note that this zone number is the “absolute” zone, not the plasma zone number. If the zone number is not specified, the line number is used as the zone number.

```
"defineback.f" 5.5 ≡
  @m format_max LINELEN    // format_code is parsed from a line
      // Reserve first 9 values for background indices
  @m form_undefined 0
  @m form_zone 10
  @m form_density 11
  @m form_temperature 12
  @m form_velocity_1 13
  @m form_velocity_2 14
  @m form_velocity_3 15

  @m data_loop #:0
  @m kount_loop #:0    // If “count_loop”, will assume it’s a comment

⟨ Read Plasma From File 5.5 ⟩ ≡
  subroutine get_n_t(nt_string)

    implicit_none_f77
    bk_common    // Common
    zn_common
    implicit_none_f90

    character*FILELEN nt_string    // Input

    integer p, b, e, length, count, line_num,    /* Local */
      sub_count, open_stat, diskin3
    integer format_code format_max
    character*LINELEN line, plasma_format
    character*FILELEN plasma_file

    zn_decl(zone)
    bk_decl(back)

    ⟨ Memory allocation interface 0 ⟩
    st_decls

    p = 0
    assert(next_token(nt_string, b, e, p))
    plasma_file = nt_string(b : e)
    diskin3 = diskin + 2
    open_file(diskin3, plasma_file)
    assert(read_string(diskin3, line, length))
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))    /* First line of the file must be a format string */
    plasma_format = line(: length)
    call parse_format(plasma_format, bk_num, format_code, count)
    assert(count > 0)
```

```

    line_num = 0
data_loop: continue
    if ( $\neg$ read_string(diskin3, line, length))
        goto eof
    line_num++
    zone = line_num
    assert(length  $\leq$  len(line))
    length = parse_string(line(: length))
    p = 0
    sub_count = 0
kount_loop: continue
    sub_count++
    if (sub_count > count)
        goto data_loop
    assert(next_token(line, b, e, p))
    if (format_code_sub_count  $\equiv$  form_zone) then
        zone = read_integer(line(b : e))
        assert(zn_check(zone))
        assert(zn_type(zone)  $\equiv$  zn_plasma)
    else if (format_code_sub_count  $\equiv$  form_density) then
        sub_count++
        back = format_code_sub_count
        assert(bk_check(back))
        assert(zn_check(zone))
        assert(zn_type(zone)  $\equiv$  zn_plasma)
        bk_n(back, zone) = read_real(line(b : e))
        assert(bk_n(back, zone)  $\geq$  zero)
    else if (format_code_sub_count  $\equiv$  form_temperature) then
        sub_count++
        back = format_code_sub_count
        assert(bk_check(back))
        assert(zn_check(zone))
        assert(zn_type(zone)  $\equiv$  zn_plasma)
        bk_temp(back, zone) = read_real(line(b : e)) * electron_charge
        assert(bk_temp(back, zone)  $\geq$  zero)
    else if (format_code_sub_count  $\equiv$  form_velocity_1) then
        sub_count++
        back = format_code_sub_count
        assert(bk_check(back))
        assert(zn_check(zone))
        assert(zn_type(zone)  $\equiv$  zn_plasma)
        bk_v(back, zone)1 = read_real(line(b : e))
    else if (format_code_sub_count  $\equiv$  form_velocity_2) then
        sub_count++
        back = format_code_sub_count
        assert(bk_check(back))
        assert(zn_check(zone))
        assert(zn_type(zone)  $\equiv$  zn_plasma)
        bk_v(back, zone)2 = read_real(line(b : e))
    else if (format_code_sub_count  $\equiv$  form_velocity_3) then
        sub_count++

```

```
      back = format_code_sub_count
      assert(bk_check(back))
      assert(zn_check(zone))
      assert(zn_type(zone) ≡ zn_plasma)
      bk_v(back, zone)_3 = read_real(line(b : e))
end if
goto kount_loop

eof: continue
      close(unit = disk3)

      return
end
```

See also section 5.6.

This code is used in section 5.4.

Parse format string. This routine takes the text description of the plasma data format provided by the user and translates it into an integer code that can be more readily used in reading the data. This additional step also facilitates testing the format. One additional capability simplifies the specification of data for a large number of background species.

```
"defineback.f" 5.6 ≡
  @m token_loop #:0

⟨ Read Plasma From File 5.5 ⟩ +≡
  subroutine parse_format(format_string, bk_num, format_code, count)

    implicit_none_f77
    implicit_none_f90

    character*LINELEN format_string    // Input
    integer bk_num

    integer count    // Output
    integer format_code format_max

    integer p, b, e, length, b_repeat, b_code, bi, sub_count, i, j    // Local
    character*LINELEN line

    ⟨ Memory allocation interface 0 ⟩
    st_decls

    line = format_string
    length = parse_string(line)
    p = 0
    count = 0
    b_repeat = FALSE
    /* If the number of background species exceeds 10, need to do this more intelligently. */
    assert(bk_num < 10)

token_loop: continue
  if (¬next_token(line, b, e, p))
    goto eof
  count++
  if ((line(b : e) ≡ 'zone') ∨ (line(b : e) ≡ 'ZONE')) then
    format_code_count = form_zone
  else
    if ((line(b : b) ≡ 'n') ∨ (line(b : b) ≡ 'N')) then
      format_code_count = form_density
    else if ((line(b : b) ≡ 't') ∨ (line(b : b) ≡ 'T')) then
      format_code_count = form_temperature
    else if ((line(b : b) ≡ 'v') ∨ (line(b : b) ≡ 'V')) then
      if (line(b + 1 : b + 1) ≡ '1') then
        format_code_count = form_velocity_1
      else if (line(b + 1 : b + 1) ≡ '2') then
        format_code_count = form_velocity_2
      else if (line(b + 1 : b + 1) ≡ '3') then
        format_code_count = form_velocity_3
      else
        write(stderr, *) 'Illegal format token, ', line(b : e)
        assert(F)
      end if
    end if
  end if
```

```

else
  write(stderr, *) 'Illegal_format_token,', line(b:e)
  assert( $\mathcal{F}$ )
end if  /* Note: had originally wanted this test to look like index(line(b:e), '(') > 0, but
        had trouble getting ftangle to deal with it (the array index switching option is to blame). The
        workaround is a bit heavy-handed but effective. Here's John Krommes' explanation:

```

This handles the entire first argument as a string, essentially defeating the index-reversing feature. So that it doesn't get confused by the comma for figuring out the first argument to \$TRANSLIT, I've replaced the comma by *C*; it then translates it back.

```

*/
count++
b_code = $TRANSLIT(index(line(b:e)C'('), 'C', ',') + 1
assert(b_code > 1)
b_code = b + b_code - 1  /* Relate to start of whole line
if (line(b_code : b_code)  $\equiv$  'b'  $\vee$  line(b_code : b_code)  $\equiv$  'B') then
  b_repeat = TRUE
  format_code_count = 1
else
  assert(b_repeat  $\equiv$  FALSE)
  bi = read_int_soft_fail(line(b_code : b_code))
  if (bi  $\equiv$  int_undef) then
    write(stderr, *) 'Bad_background_code_in_token', line(b:e)
    assert( $\mathcal{F}$ )
  end if
  format_code_count = bi
end if
end if
goto token_loop
eof: continue
assert(count > 0)
if ((b_repeat  $\equiv$  TRUE)  $\wedge$  (bk_num > 1)) then
  sub_count = count
  do i = 2, bk_num
    do j = 1, sub_count
      if ((format_code_j  $\equiv$  form_density)  $\vee$  (format_code_j  $\equiv$  form_temperature)  $\vee$  (format_code_j  $\equiv$ 
        form_velocity_1)  $\vee$  (format_code_j  $\equiv$  form_velocity_2)  $\vee$  (format_code_j  $\equiv$  form_velocity_3))
        then
          count++
          format_code_count = format_code_j
          /* If not a zone number, then only other possibility is a background number. */
        else if (format_code_j  $\neq$  form_zone) then
          count++
          format_code_count = i
        end if
      end do
    end do
  end do
end if
return
end

```

Compute background densities and temperatures for NCSX.

```
"defineback.f" 5.7 ≡
  @m max_stratum 100
  @m max_radaii 100

  @m radii_done #:0
  @m nT_done #:0

⟨ Plasma Data For NCSX 5.7 ⟩ ≡
  subroutine get_n_t(nt_string)

    define_varp(zone_stratum, INT, zone_ind)

    implicit_none_f77
    zn_common // Common
    sp_common
    pr_common
    bk_common
    g2_common
    implicit_none_f90

    character*LINLEN nt_string // Input

    integer is, ix, p, b, e, length, stratum, zone, j, num_x, // Local */
    close_zone, open_stat, lcms_stratum, fileid, zone2, back, ipoly, disk3

    real x_in, jx_in, delta_min, delta, lambda_n, lambda_t
    real stratum_radius max_stratum, r_over_a_0:max_radaii, ne_0:max_radaii, te_0:max_radaii, ti_0:max_radaii

    character*LINLEN line
    character*FILELEN radii_file, polygon_file, nT_profile

    vc_decl(vec_delta)

    external find_index, interpolate1 // External
    real find_index, interpolate1

    declare_varp(zone_stratum)

    ⟨ Memory allocation interface 0 ⟩
    st_decls
    vc_decls
    g2_ncdecl
    nc_decls

    var_alloc(zone_stratum) /* Could have put all of these arguments in yet another text input file,
    but decided that there were already too many input files for this routine. For clarity, the order
    of the arguments is: radii_file nT_profile polygon_file lambda_n lambda_t */

    p = 0
    assert(next_token(nt_string, b, e, p))
    radii_file = nt_string(b : e)

    assert(next_token(nt_string, b, e, p))
    nT_profile = nt_string(b : e)

    assert(next_token(nt_string, b, e, p))
    polygon_file = nt_string(b : e)

    assert(next_token(nt_string, b, e, p))
    lambda_n = read_real(nt_string(b : e))
```

```

    assert(lambda_n > zero)
    assert(next_token(nt_string, b, e, p))
    lambda_t = read_real(nt_string(b : e))
    assert(lambda_t > zero)

    disk3 = disk3 + 2
    open_file(disk3, radii_file)
    assert(read_string(disk3, line, length))    // Column header
    do is = 1, max_stratum
        stratum_radius_is = real_unused
    end do
    do is = 1, max_stratum
        if (read_string(disk3, line, length)) then
            assert(length ≤ len(line))
            length = parse_string(line(: length))
            p = 0
            assert(next_token(line, b, e, p))
            stratum = read_integer(line(b : e))
            assert(stratum ≤ max_stratum)
            assert(stratum_radius_stratum ≡ real_unused)
            assert(next_token(line, b, e, p))
            stratum_radius_stratum = read_real(line(b : e))
            assert(stratum_radius_stratum ≥ zero ∧ stratum_radius_stratum ≤ one)
        else
            goto radii_done
        end if
    end do
    assert('Stratum_radius_file_too_long' ≡ '_')
radii_done: continue    /* Assume that these radii are defined only over the LCMS and that the last one
                        represents the LCMS itself. */
    lcms_stratum = stratum
    close(unit = disk3)

    do zone = 1, zn_num
        zone_stratum_zone = int_unused
    end do

    assert(polygon_file ≠ char_undef)
    fileid = ncopen(polygon_file, NC_NOWRITE, nc_stat)
    g2_ncread(fileid)

    do ipoly = 1, g2_num_polygons
        zone = g2_polygon_zone_ipoly
        if (zone_stratum_zone ≡ int_unused) then
            assert(g2_polygon_stratum_ipoly ≥ 0)    // Do we need this?
            zone_stratum_zone = g2_polygon_stratum_ipoly
        else    // Or this?
            assert(zone_stratum_zone ≡ g2_polygon_stratum_ipoly)
        end if
    end do

    open_file(disk3, nT_profile)
    assert(read_string(disk3, line, length))    // Column header /* Begin these arrays at zero so
        they will be consistent with the interpolation routines.
    do ix = 0, max_radii

```



```

    r_over_a_ix = real_unused
    ne_ix = real_unused
    te_ix = real_unused
    ti_ix = real_unused
end do
do ix = 0, max_radii
  if (read_string(diskin3, line, length)) then
    assert(length ≤ len(line))
    length = parse_string(line(: length))
    p = 0
    assert(next_token(line, b, e, p))
    j = read_integer(line(b : e))
    assert(j ≡ ix + 1)    // Be sure arrays ordered correctly
    assert(next_token(line, b, e, p))
    r_over_a_ix = read_real(line(b : e))
    assert(r_over_a_ix ≥ zero ∧ r_over_a_ix ≤ one)
    assert(next_token(line, b, e, p))
    /* Assume density is in units of 1019 m-3 and temperatures in keV. */
    ne_ix = read_real(line(b : e)) * const(1., 19)
    assert(ne_ix > zero)
    assert(next_token(line, b, e, p))
    te_ix = read_real(line(b : e)) * const(1., 3) * electron_charge
    assert(te_ix > zero)
    assert(next_token(line, b, e, p))
    ti_ix = read_real(line(b : e)) * const(1., 3) * electron_charge
    assert(ti_ix > zero)
  else
    goto nT_done
  end if
end do
assert('Plasma_profile_file_too_long' ≡ ' ')
nT_done: continue
num_x = j    // Last value read /* Done reading files. Interpolate density and temperature values
           for those zones inside the LCMS.
do back = 1, bk_num
  assert(bk_check(back))
  do zone = 1, zn_num
    assert(zn_check(zone))
    if (zn_type(zone) ≡ zn_plasma) then
      if (zone_stratum_zone ≤ lcms_stratum) then
        x_in = stratum_radius_zone_stratum_zone
        jx_in = find_index(x_in, r_over_a, num_x)
        if (sp_sy(pr_background(back)) ≡ 'e') then
          bk_n(back, zone) = interpolate1(jx_in, ne, num_x)
          bk_temp(back, zone) = interpolate1(jx_in, te, num_x)
        else
          bk_n(back, zone) = interpolate1(jx_in, ne, num_x)
          bk_temp(back, zone) = interpolate1(jx_in, ti, num_x)
        end if
      else
        delta_min = geom_infinity
        do zone2 = 1, zn_num

```

```

/* Find those zones representing the LCMS. Calculate minimum distance from zone to
   one of them. Compute density and temperature using exponential fall-off lengths. */
if (zone_stratumzone2  $\equiv$  lcms_stratum) then
  vc_difference(zone_centerzone, zone_centerzone2, vec_delta)
  delta = vc_abs(vec_delta)
  if (delta < delta_min) then
    delta_min = delta
    close_zone = zone2
  end if
end if
end do
assert(delta_min > zero  $\wedge$  delta_min < geom_infinity)
assert(bk_n(back, close_zone) > zero)
assert(bk_temp(back, close_zone) > zero)
bk_n(back, zone) = bk_n(back, close_zone) * exp( $-\delta_{min} / \lambda_{n}$ )
bk_temp(back, zone) = bk_temp(back, close_zone) * exp( $-\delta_{min} / \lambda_{n}$ )
end if
end if
end do
end do
return
end

```

This code is used in section 5.4.

6 INDEX

all_segments: 5.2, 5.3.
aname: 5.2.
area: [5.2](#), 5.3.
arg_count: 5.1.
assert: 5.1, 5.2, 5.3, 5.5, 5.6, 5.7.
aunit: 5.2.

b: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
b_code: [5.6](#).
b_repeat: [5.6](#).
back: 5.2, 5.5, [5.7](#).
back_label: [5.2](#).
background_coords: 5.2.
bi: [5.6](#).
bk_check: 5.5, 5.7.
bk_common: 5.2, 5.5, 5.7.
bk_decl: 5.2, 5.5.
bk_n: 5.2, 5.5, 5.7.
bk_num: 5.2, 5.5, [5.6](#), 5.7.
bk_temp: 5.2, 5.3, 5.5, 5.7.
bk_v: 5.5.

cartesian: 1.
char_undef: 5.7.
circum: [5.2](#), 5.3.
close_zone: [5.7](#).
command_arg: 5.1.
const: 5.3, 5.7.
count: [5.5](#), [5.6](#).

data_loop: [5.5](#).
declare_varp: 5.2, 5.7.
DEFAULT: [5.2](#), 5.4.
define_dimen: 5.2.
define_varp: 5.2, 5.7.
defineback: [5.1](#).
definegeometry2d: 1.
delta: [5.7](#).
delta_min: [5.7](#).
density: [5.2](#).
dim_segments: [5.2](#).
diskin: 5.2, 5.5, 5.7.
diskin2: [5.2](#).
diskin3: [5.5](#), [5.7](#).
dummy: [5.2](#).

e: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
electron_charge: 5.2, 5.5, 5.7.
end_source_group: 1.
eof: 5.5, 5.6.
erase_geometry: 5.1.

exp: 5.7.
FALSE: 5.2, 5.6.
file: 5.2.
FILE: [1](#).
fileid: [5.7](#).
FILELEN: 5.1, 5.2, 5.5, 5.7.
find_index: [5.7](#).
FLOAT: 5.2.
form: 5.2.
form_density: [5.5](#), 5.6.
form_temperature: [5.5](#), 5.6.
form_undefined: [5.5](#).
form_velocity_1: [5.5](#), 5.6.
form_velocity_2: [5.5](#), 5.6.
form_velocity_3: [5.5](#), 5.6.
form_zone: [5.5](#), 5.6.
format_code: [5.5](#), [5.6](#).
format_max: [5.5](#), 5.6.
format_string: [5.6](#).
ftangle: 5.6.
geom: [5.2](#), 5.3.
geom_infinity: 5.7.
GEOMETRY: [5.2](#), 5.4.
geometry_symmetry: 5.2, 5.3.
geometry_symmetry_cylindrical: 5.2, 5.3.
geometry_symmetry_plane: 5.2, 5.3.
get_n_t: 1, 5.2, [5.5](#), [5.7](#).
gi_common: 5.2.
g2_common: 5.7.
g2_ncdecl: 5.7.
g2_ncread: 5.7.
g2_num_polygons: 5.7.
g2_polygon_stratum: 1, 5.7.
g2_polygon_zone: 1, 5.7.
i: [5.2](#), [5.6](#).
implicit_none_f77: 5.1, 5.2, 5.5, 5.6, 5.7.
implicit_none_f90: 5.1, 5.2, 5.5, 5.6, 5.7.
inc_ion_energy: 5.3.
increment_iseg: [5.2](#).
index: 5.6.
init_wt_alias: 5.2.
input_done: [5.2](#).
inputfile: [5.1](#), [5.2](#).
INT: 5.2, 5.7.
int_undef: 5.2, 5.3, 5.6.
int_uninit: 5.2.
int_unused: 5.2, 5.7.
interpolate1: [5.7](#).
iostat: 5.2.
ipoly: [5.7](#).
is: [5.7](#).

iseg: [5.2](#), [5.3](#).
ix: [5.7](#).
j: [5.6](#), [5.7](#).
jx_in: [5.7](#).
kount_loop: [5.5](#).
lambda_n: [5.7](#).
lambda_t: [5.7](#).
lcms_stratum: [5.7](#).
len: [5.2](#), [5.5](#), [5.7](#).
length: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
line: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
line_num: [5.5](#).
LINELEN: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
max_radai: [5.7](#).
max_stratum: [5.7](#).
mem_check: [5.1](#).
nargs: [5.1](#).
nc_decls: [5.7](#).
NC_NOWRITE: [5.7](#).
nc_read_materials: [5.1](#).
nc_read_pmi: [5.1](#).
nc_read_problem: [5.1](#).
nc_read_reactions: [5.1](#).
nc_read_species: [5.1](#).
nc_stat: [5.7](#).
ncopen: [5.7](#).
NCSX: [5.2](#), [5.4](#).
ne: [5.7](#).
neutralize_species: [5.3](#).
new_source_group: [1](#).
next_line: [5.2](#).
next_segment: [5.2](#).
next_srcline: [5.2](#).
next_stratum: [5.2](#).
next_strength: [5.2](#).
next_token: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
nflights: [5.2](#), [5.3](#).
nsectors: [5.3](#).
nT_done: [5.7](#).
nT_profile: [5.7](#).
nt_string: [5.2](#), [5.5](#), [5.7](#).
num_segments: [5.2](#), [5.3](#).
num_x: [5.7](#).
nunit_n: [5.2](#).
nunit_t: [5.2](#).
one: [5.3](#), [5.7](#).
open_file: [5.2](#), [5.5](#), [5.7](#).
open_stat: [5.2](#), [5.5](#), [5.7](#).
outputbrowser: [5.2](#).
p: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
parse_format: [5.5](#), [5.6](#).
parse_string: [5.2](#), [5.5](#), [5.6](#), [5.7](#).
PI: [5.3](#).
plasma_coords: [1](#).
plasma_coords_cartesian: [5.2](#).
plasma_coords_cylindrical: [5.2](#).
plasma_file: [1](#), [5.5](#).
plasma_format: [5.5](#).
plate: [1](#).
polygon_file: [5.7](#).
pr_background: [5.3](#), [5.7](#).
pr_common: [5.2](#), [5.7](#).
puff: [1](#).
r_over_a: [5.7](#).
radai_done: [5.7](#).
radai_file: [5.7](#).
read_geometry: [5.1](#).
read_int_soft_fail: [5.6](#).
read_integer: [5.2](#), [5.5](#), [5.7](#).
read_real: [5.2](#), [5.5](#), [5.7](#).
read_string: [5.2](#), [5.5](#), [5.7](#).
readfilenames: [5.1](#).
real_uninit: [5.2](#), [5.3](#).
real_unused: [5.7](#).
root_sp: [5.2](#), [5.3](#).
sc_common: [5.2](#).
sc_decl: [5.2](#).
sc_neg: [5.3](#).
sc_pos: [5.3](#).
sc_target: [5.3](#).
sc_target_check: [5.3](#).
sector: [5.2](#), [5.3](#).
sector_points: [5.3](#).
sector_strata_segment: [5.3](#).
sector_type_pointer: [5.3](#).
sector_zone: [5.3](#).
segment_ind: [5.2](#).
segments_done: [5.2](#).
set_background_sources: [5.2](#).
setup_back_arrays: [5.1](#).
setup_background: [5.1](#), [5.2](#).
so_base: [5.3](#).
so_chkpt: [5.3](#).
so_common: [5.2](#).
so_geom: [5.3](#).
so_grps: [5.2](#), [5.3](#).
so_line: [5.2](#).
so_name: [5.2](#).
so_nflights: [5.3](#).
so_nseg: [5.3](#).

so_plate: 5.3.
so_point: 5.2.
so_root_sp: 5.3.
so_scale: 5.3.
so_seg_tot: 5.2, 5.3.
so_set_run_flags: 5.2.
so_species: 5.3.
so_surface: 5.2.
so_tot_curr: 5.3.
so_type: 5.3.
so_type_num: 5.2.
so_volume: 5.2.
source_base_ptr: 5.2, 5.3.
source_current: 5.2, 5.3.
source_file: 1, 5.2.
source_geom: 1.
source_geometry: 5.2, 5.3.
source_nflights: 1.
source_num_checkpoints: 5.2, 5.3.
source_num_flights: 5.2, 5.3.
source_num_segments: 5.2, 5.3.
source_root_sp: 1.
source_root_species: 5.2, 5.3.
source_scale_factor: 5.2, 5.3.
source_segment: 1, 5.2, 5.3.
source_segment_prob_alias: 5.2, 5.3.
source_segment_ptr: 5.2, 5.3.
source_segment_ptr_alias: 5.2, 5.3.
source_segment_rel_wt: 5.2, 5.3.
source_species: 1, 5.2, 5.3.
source_stratum: 1, 5.2, 5.3.
source_strength: 1, 5.2, 5.3.
source_total_current: 5.2, 5.3.
source_type: 1, 5.2, 5.3.
source_weight_norm: 5.2, 5.3.
sp_check: 5.2, 5.3.
sp_common: 5.2, 5.7.
sp_decl: 5.2.
sp_lookup: 5.2.
sp_sy: 5.3, 5.7.
species: 5.2, 5.3.
species_infile: 1.
specify_current: 1.
specify_flux: 1, 5.2, 5.3.
srcfile_done: 5.2.
st_decls: 5.2, 5.5, 5.6, 5.7.
status: 5.2.
stderr: 5.6.
strata: 1, 5.3.
strata_done: 5.2.
stratum: 1, 5.7.
stratum_radius: 5.7.

strengths_done: 5.2.
sub_count: 5.5, 5.6.
sy_decls: 5.1.
targ: 5.2, 5.3.
target_e_ion_delta: 1, 5.3.
target_e_ion_mult: 1, 5.3.
te: 5.7.
temperature: 5.2.
ti: 5.7.
token_loop: 5.6.
TRUE: 5.2, 5.3, 5.6.
type: 5.2, 5.3.
type_string: 5.2.
uedge: 5.2.
unit: 5.2, 5.5, 5.7.
universal_cell_max: 5.3.
universal_cell_min: 5.3.
var_alloc: 5.7.
var_free: 5.2.
var_realloca: 5.2, 5.3.
var_reallocb: 5.2.
vc_abs: 5.3, 5.7.
vc_decl: 5.2, 5.7.
vc_decls: 5.7.
vc_difference: 5.3, 5.7.
vec_delta: 5.7.
write_background: 5.2.
write_geometry: 5.2.
x_in: 5.7.
xdiff: 5.2, 5.3.
zero: 5.2, 5.3, 5.5, 5.7.
zn_check: 5.5, 5.7.
zn_common: 5.2, 5.5, 5.7.
zn_decl: 5.2, 5.5.
zn_num: 5.2, 5.7.
zn_plasma: 5.2, 5.5, 5.7.
zn_type: 5.2, 5.5, 5.7.
zone: 5.2, 5.3, 5.5, 5.7.
zone_center: 5.7.
zone_ind: 5.7.
zone_stratum: 1, 5.7.
zone2: 5.7.

〈Functions and subroutines 5.2, 5.4〉 Used in section 5.1.
〈Memory allocation interface 0〉 Used in sections 5.7, 5.6, 5.5, and 5.2.
〈Plasma Data For NCSX 5.7〉 Used in section 5.4.
〈Read Plasma From File 5.5, 5.6〉 Used in section 5.4.
〈Setup Source Group 5.3〉 Used in section 5.2.

COMMAND LINE: "fweave -f -i! -W[-ykw700 -ytw40000 -j -n/
/u/dstotler/degas2/src/defineback.web".

WEB FILE: "/u/dstotler/degas2/src/defineback.web".

CHANGE FILE: (none).

GLOBAL LANGUAGE: FORTRAN.